

ResQ — AI-Assisted Disaster Relief Coordination

A LangGraph multi-agent system for Himachal Pradesh: one incident report in — an enriched, ranked, explainable, human-approved response plan out.

6-agent LangGraph pipeline

RAG over 6 official HP datasets

Explainable urgency scoring

Human-approved email dispatch

Major Capstone Project · Applied AI

Team ResQ · Python · Streamlit · LangGraph · LangChain · Ollama · ChromaDB

The Problem: Disaster Triage Is an Information Problem

Himachal Pradesh — flash floods, landslides, cloudbursts, GLOF, wildfire

What responders face

- Affected people need Medical, Shelter, Food, Water, Rescue, Transport — all at once, from one location report.
- Kangra alone logged 4,027 landslides in 2023; Mandi 2,169 (HIMCOSTE inventory).
- Key NH corridors (NH-154 @ Kotrupi, NH-5 @ Nigulsari...) close chronically every monsoon — routes can't be assumed.
- Hospitals, NULM shelters, schools, CWC river stations and volunteers live in disconnected PDFs, CSVs and spreadsheets.

What control rooms actually need

- A ranked shortlist of actionable resources — not a raw database dump.
- Context: live weather alert, district risk tier, GLOF & wildfire proneness.
- A route estimate with known road-risk warnings.
- An explainable urgency score to prioritise between incidents.
- An auditable, human-approved way to actually contact providers.

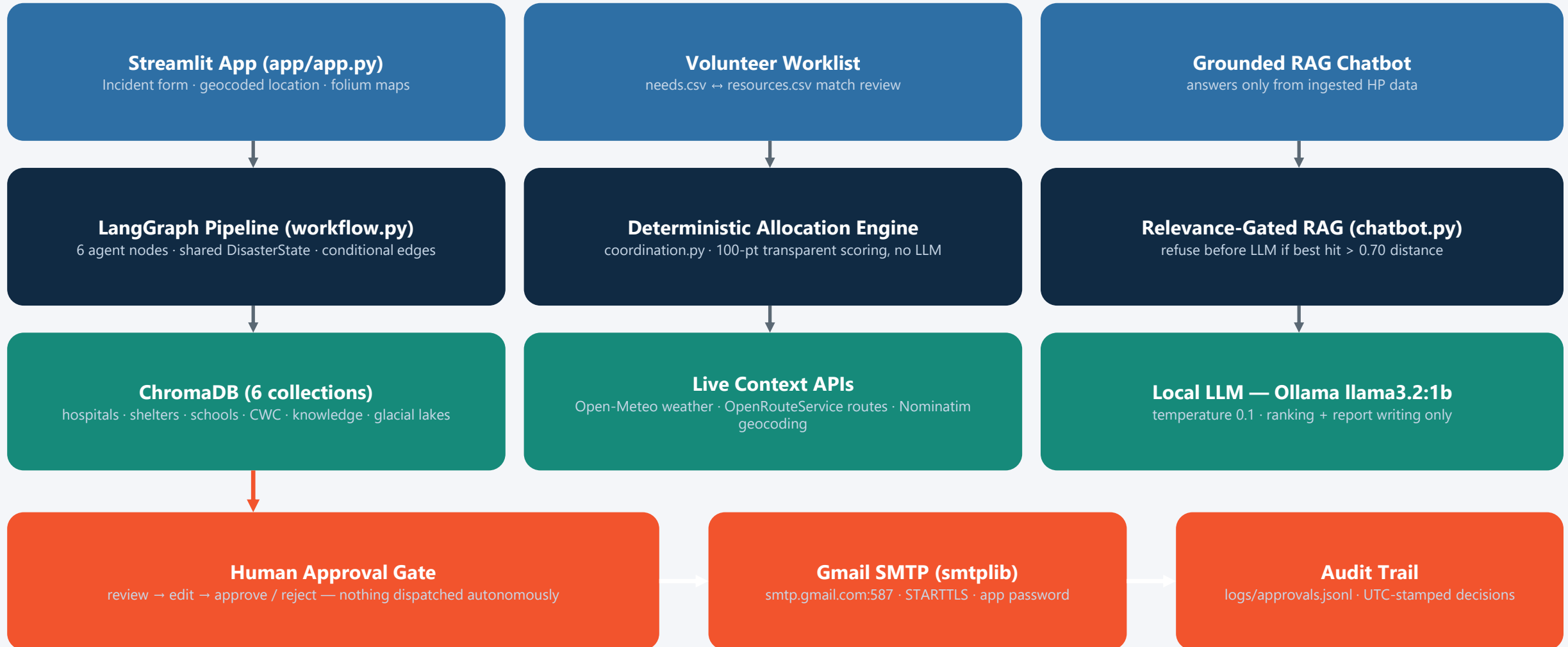
Our answer

ResQ ingests one structured incident report (district, place, disaster type, needs), enriches it with live weather and risk intelligence, retrieves and ranks matching resources, plans a route, computes an explainable 0–100 urgency score, and drafts a coordination email — which a human reviews, edits and approves before anything is sent.

Design stance: decision support, never an autonomous dispatcher. Every output says “verify with local authorities”; every send is gated by a person and logged to an audit trail.

System Architecture — Four Layers, One Audit Trail

Streamlit UI → LangGraph agents → retrieval & live context → human approval gate



hp_hospitals

289 NHP hospitals (CSV) — district, type, contact, specialities

hp_shelters

DAY-NULM urban shelters (PDF) — city, capacity

hp_schools

Govt schools 2021 (PDF) — backup shelter proxies

hp_cwc_stations

CWC flood-forecast stations (Excel) — river, coordinates

hp_disaster_knowledge

HIMCOSTE 2023 landslide inventory + NDMA/IMD guidance

hp_glacial_lakes

CWC Sept-2025 GLOF monitoring — extracted from the PDF by build_glacial_lakes_csv.py

Embedding & retrieval choices

- sentence-transformers all-MiniLM-L6-v2 — 384-dim, runs locally, fast enough to embed the full corpus in minutes.
- ChromaDB PersistentClient — an embedded vector DB: no server to run, survives restarts in chroma_db/.
- Metadata filters (where district = KULLU) give exact district scoping on top of semantic search — with an unfiltered fallback if a district returns nothing.
- python scripts/ingest.py --overwrite rebuilds everything reproducibly.

Two datasets stay outside the vector store — on purpose

- VIIRS wildfire hotspot history (CSV) → loaded as numpy arrays; proneness is a vectorised haversine density count, not a semantic query.
- needs.csv / resources.csv (volunteer pool) → pandas records; allocation must be exact-match arithmetic, not fuzzy retrieval.

Principle: *embeddings where language matters, plain math where numbers matter.*

The LangGraph Workflow — a 6-Node State Machine

workflow.py · every node reads & writes one shared DisasterState (TypedDict, ~30 fields)

1 · intake_agent

weather + IMD alert · district risk ·
wildfire flag

2 · glof_monitor

expanding glacial lakes → WATCH
/ ADVISORY

3 · resource_finder

ChromaDB: hospitals · shelters ·
CWC stations

4 · matching_agent

LLM ranks resources → priority
pick (JSON)

5 · route_planner

ORS route + NH road-risk
warnings

6 · escalation_agent

urgency score · final report ·
contacts

conditional edge: no resource found → skip routing, escalate

Why LangGraph (and not CrewAI / a plain chain)

- Genuine conditional branches: flood → CWC river checks; no match → straight to escalation. Sequential task chaining hides this.
- State must persist and accumulate: weather found in node 1 is still needed by the report in node 6 — one typed DisasterState carries it.
- `add_conditional_edges` makes the branch logic explicit, testable code — `should_skip_to_escalation()` is a 4-line pure function.
- A `disaster_type_router` edge is already wired in — future disaster-specific sub-agents plug in without rewiring.
- Human-in-the-loop attaches naturally after the terminal node.

State design details that pay off

- DisasterState is a TypedDict — every field a node fills is declared: inputs, enriched context, resources, matches, route, report.
- `node_log: Annotated[list[str], operator.add]` — each node appends a ✓/△ line; LangGraph merges them into a built-in agent trace shown in the UI.
- Nodes return `{**state, ...updates}` — pure functions of state, so each is unit-testable in isolation (18 pytest scenarios).
- `graph.compile()` yields an app; `run_agent()` seeds the initial state and invokes it end-to-end.

intake_agent — enrich the report

Weather → alert: Open-Meteo 24h precipitation is mapped to IMD-style levels:

RED ≥ 204.5 mm · **ORANGE** ≥ 115.6 · **YELLOW** ≥ 64.5 · **GREEN** below

District risk: HIMCOSTE 2023 landslide counts → CRITICAL/HIGH/MEDIUM/LOW tier per district, with key rivers.

Nearest CWC station: haversine scan over all river-monitoring stations → link to live flood forecasts.

Wildfire proneness: vectorised numpy haversine over every historical VIIRS fire hotspot; count within 10 km →

HIGH ≥ 100 hotspots · **MODERATE** ≥ 25 · **LOW** ≥ 1 · **MINIMAL** 0

Knowledge chunks: RAG query for district + disaster-type guidance feeds later prompts.

Resilience: if the weather API fails, a clearly-labelled MOCK fallback keeps the pipeline alive.

glof_monitor_agent — early warning for a Himalayan killer

- Glacial Lake Outburst Floods gave HP some of its worst recent disasters — so GLOF gets a dedicated node.

- Queries the hp_glacial_lakes collection: lakes ranked by (same district, expanding status, proximity).

Any lake with an increasing water-spread area triggers an alert:

WATCH if the disaster is water-driven (GLOF, Flash Flood, Cloudburst)

ADVISORY otherwise — the hazard exists but isn't the active event

- The alert message names the nearest expanding lake, its basin/river, distance, and % area change — e.g. "+9% water-spread, ~14 km away, elevated GLOF risk downstream".

- Honesty baked in: every alert carries a disclaimer that it is based on CWC's previous-year monthly satellite monitoring (Sep 2025), not real-time water levels.

- Validation: the extraction pipeline found 12 expanding HP lakes — exactly CWC's own published figure.

resource_finder_agent — retrieval driven by need & disaster

Medical need or injury-prone disaster → top-5 hospitals (district-filtered semantic query)

Shelter need or displacement disaster → NULM shelters + govt schools as backup shelters

Flash Flood / GLOF / Cloudburst → CWC stations on the nearest river for live-level lookup

- Each query combines semantic text ("hospital emergency KULLU...") with a hard metadata filter on district — and falls back to an unfiltered query if the district yields nothing.
- Also pulls 3 extra knowledge chunks ("**<disaster> response resources <district>**") and merges them into state for the prompts downstream.

Node log: "✓ found 5 hospitals, 3 shelters, 2 CWC stations" — the trace users see.

matching_agent — LLM ranking with a deterministic parachute

- All retrieved hospitals + shelters are pooled and tagged with resource_type.

Empty pool? Set escalation_needed=True and return — the conditional edge then skips routing entirely.

- Otherwise llama3.2:1b gets the situation (district, disaster, needs, IMD alert, risk tier), the full resource JSON, and knowledge chunks...

...and must answer in a strict JSON contract: **priority_resource · ranked_resources · 2-sentence reasoning**

- Temperature 0.1 keeps it near-deterministic; the reply is stripped of ``` fences and json.loads-ed.

If parsing fails: fall back to retrieval order — the top semantic match becomes priority, and the reasoning says a fallback was used. The pipeline never crashes on a bad LLM reply.

Pattern: LLM for judgement, code for guarantees.

route_planning_agent — a destination ladder that never fails

Resolve where the resource actually is, best-effort:

resource coords → geocode full name → geocode the town inside the name (“Civil Hospital Dalhousie” → “Dalhousie”) → district centre

- Routing: OpenRouteService turn-by-turn; if ORS is down/keyless → straight-line haversine at ~30 km/h mountain speed, explicitly labelled approximate.
- Computes routes to the top hospital AND top shelter, plus NH corridor warnings (Kotrupi, Nigulsari...) during monsoon months.

escalation_agent — the report the human reads

- llama3.2:1b writes a markdown situation report from the full state: weather, risks, priority resource, route, GLOF/wildfire flags, top-3 alternatives.
- Always appends HP emergency contacts (NDMA 1078, Police 100, 108, NDRF...) and the nearest hospital's phone number.
- Every report ends: “AI-assisted information — always verify with local authorities.”

Explainable urgency score (0–100) — pure arithmetic, no LLM

IMD alert RED 35 · ORANGE 25 · YELLOW 15 · GREEN 5

District tier CRITICAL 25 · HIGH 18 · MEDIUM 10 · LOW 5

Needs (top 2) Rescue/Medical 10 · Evacuation 9 · Shelter/Water 6 · Food 4

GLOF signal WATCH 12 · ADVISORY 6

Wildfire HIGH 10 · MODERATE 6 · LOW 2

Escalation flag +10 when no resource could be matched

Σ capped at 100 → CRITICAL ≥ 75 · HIGH ≥ 50 · MODERATE ≥ 30 · LOW

The per-factor breakdown ships with the score, so a coordinator can see exactly why one incident outranks another — and challenge it.

Category match 40

Location overlap ≤ 25

Quantity coverage ≤ 20

Available 8

Verified 7

100-point match score — every point traceable to a rule, so two coordinators always get the same allocation

How a request becomes an allocation

1. Requests arrive from needs.csv or free text/tweets — a rule-based extractor pulls category, urgency, quantity and location from messages like “30 people trapped near Aut, need rescue urgently”.
2. Category is a hard filter — a Food provider can never be allocated to a Medical need.
3. Location score = token overlap of place names (Jaccard) · quantity score = $\min(\text{available}/\text{needed}, 1)$.
4. Ties break on provider urgency-capacity; each need keeps its best match + 2 alternatives.
5. The worklist is sorted by need urgency (Critical first), unmatched needs surfaced first within each level — triage order, not file order.

Allocation outcomes

MATCHED score ≥ 55 — draft outreach to the provider for $\min(\text{needed}, \text{available})$ units.

PARTIAL best match exists but is weak/short — coverage % and the exact unit shortfall are shown, with “consider a second provider”.

UNMATCHED nothing eligible — an escalation draft to HPSDMA 1077 / NDMA 1078 is generated instead. No request silently disappears.

Why no LLM here: *allocation decides who gets scarce resources. It must be transparent, reproducible and auditable — properties arithmetic has and generative models don't. The LLM assists around the decision; it never makes it.*

Validated: all 5 sample needs matched to the correct provider in tests.

10 Matching, Step by Step — One Need, Fully Scored

coordination.py · a real request from needs.csv scored against every eligible provider

Incoming need N003 Category **Food** Location **Mandi riverbank** Quantity **60 meal kits** Reported by **NGO field lead**

Same request can arrive from a tweet — the rule-based extractor pulls category / quantity / location before scoring.

Score every eligible provider — category is a hard filter, so only Food providers are scored:

Eligible Food provider	Category 40	Location ≤25	Quantity ≤20	Avail+Verif 15	Score /100
Red Cross Mandi (Mandi)	40	12.5	20	15	87.5 ✓ BEST
Annapurna Kitchen (Shimla)	40	0	20	15	75.0
Solan Wholesale Foods (Solan)	40	0	20	15	75.0

Why Red Cross Mandi wins

Category 40 hard filter passed (Food = Food). **Location 12.5** “Mandi” token overlaps (0.5 Jaccard × 25).

Quantity 20 120 kits ≥ 60 needed → full coverage. **Avail 15** Available (8) + Verified (7).

- The two Shimla/Solan providers tie at 75.0 — identical except location. Proximity is the whole difference: the same rubric, applied identically, breaks the tie.

Outcome → an actionable draft

Score 87.5 ≥ 55 → MATCHED. Draft outreach for min(60, 120) = 60 kits, coverage 100%.

- Best match + 2 ranked alternatives are kept, so a coordinator can override the pick with one click.

Every number above is reproducible: no LLM touches the allocation score — two coordinators always see the same 87.5.

Why per-need scoring isn't enough

- Scoring each need independently makes one provider with 100 units look fully available to every need at once.
 - In a real dispatch that double-promises scarce stock — two coordinators act on the same 100 units.
- Fix:** needs are processed in urgency order, and each best match reserves its units from a working copy of the pool before the next need is scored.
- So allocation reflects triage priority AND real remaining stock — not file order, not listed capacity.

One provider · 100 units · 3 competing needs

Served in urgency order, not file order:

① CRITICAL · needs 80

reserve 80 → 20 left **MATCHED**

② HIGH · needs 50

only 20 left → reserve 20 **PARTIAL**
shortfall -30 → "consider a second provider"

③ MEDIUM · needs 15

provider exhausted, skipped **UNMATCHED**
→ escalation draft to HPSDMA 1077 / NDMA 1078

committed_units never exceed what's actually available — total dispatched ≤ 100, always.

Dispatch ledger — stock stays spent

Approve & send → **log_dispatch()** appends {resource_id, units, request_id} to logs/dispatch_ledger.jsonl.

load_resources() subtracts dispatched totals: remaining = listed – dispatched, floored at 0.

- The UI shows listed / dispatched / remaining per provider, so depletion is visible across sessions.
- Approvals in one incident correctly shrink the pool for the next — later needs degrade to PARTIAL/UNMATCHED on their own.

Proven in tests: no double-promising, urgency-first order, exhaustion, and stock never goes negative.

From Match to Mailbox — the Human-in-the-Loop Email Flow

Every coordination message is drafted by the system, decided by a person, and logged forever

1 · Draft

`draft_coordination_message()` —
provider, units, coverage %, confidence
/100

2 · Review

coordinator reads the draft in Streamlit
— full text, editable

3 · Decide

✓ approve & send · ✓ approve only ·
✗ reject

4 · Send

smtplib → smtp.gmail.com:587 ·
STARTTLS · Gmail app password

5 · Audit

decision appended to
logs/approvals.jsonl (UTC timestamp)

What's in the email

- Header: "DISASTER RELIEF COORDINATION — MEDICAL [CRITICAL priority]".
- The need: request ID, quantity, location, who reported it, notes.
- The match: provider name & location, listed stock, availability + contact-verification status, estimated coverage %.
- The ask: dispatch min(needed, available) units + expected arrival time.
- The caveat: "Match confidence 87/100 — requires human coordinator approval before sending."
- Unmatched needs get a different draft: an escalation to the district control room instead of provider outreach.

Why this design

Nothing sends itself. Drafts are inert text until a person clicks approve — in a disaster, a wrong automated message costs trust and lives.

- Editable before send — the human can correct quantities or add local knowledge the system lacks.
- Approve, reject AND send-failures are all logged to `approvals.jsonl` — a replayable audit trail of every human decision.
- Plain smtplib + EmailMessage: zero vendor lock-in, works with any Gmail app password entered in the sidebar (never stored in code).
- The same gate wraps both surfaces: volunteer-worklist matches and the LangGraph agent's own coordination draft.

LangGraph	Explicit state machine: typed shared state, conditional edges, natural human-in-the-loop hook — branching CrewAI-style chains can't express.
LangChain (core + ollama)	Uniform chat-model interface & message types; swap the LLM without touching agent code.
Ollama · llama3.2:1b @ t=0.1	Fully local & free — no cloud dependency when connectivity is down (exactly when disasters strike); no sensitive location data leaves the machine.
ChromaDB	Embedded, persistent vector DB — no server to operate; metadata filters give exact district scoping on top of semantic search.
sentence-transformers (MiniLM-L6-v2)	384-dim local embeddings: fast, offline, strong enough that the RAG relevance gate separates in/out-of-scope by a 0.18 distance margin.
Streamlit	A control-room UI in pure Python; session_state drives the approve/reject workflow without a JS frontend.
pandas + numpy	CSV/Excel wrangling; vectorised haversine scores thousands of VIIRS fire hotspots in milliseconds.
folium	Interactive Leaflet maps of user, resources and routes — embedded straight into Streamlit.
Open-Meteo	Free, key-less forecast API → 24h rainfall → IMD-style alert derivation; mock fallback keeps demos alive offline.
OpenRouteService + Nominatim	Free-tier road routing and geocoding; both wrapped with labelled fallbacks (straight-line estimate, district centre).
smtplib / EmailMessage (stdlib)	Email dispatch with no third-party service — one less credential, one less point of failure.
pytest + loguru	18 deterministic scenario tests pin urgency bands, matching, and the RAG gate; structured logs for every tool failure.

Common thread: *everything critical runs locally and degrades gracefully — the agent still answers when the internet doesn't.*

Guardrails

- Human approval gates every outbound message; full decision audit in approvals.jsonl.
- RAG chatbot refuses out-of-scope questions before the LLM runs (relevance gate at 0.70 cosine distance) and is prompt-locked to retrieved context only.
- Every fallback is labelled: MOCK weather, straight-line routes, previous-year GLOF/wildfire data disclaimers.
- Every report ends with "verify with local authorities".

Measured results

- 18/18** deterministic pytest scenarios pass — urgency bands, matching, escalation, RAG gate.
- 0** false accepts/rejects at the RAG gate: in-scope ≤ 0.59 vs out-of-scope ≥ 0.77 distance.
- 12** expanding glacial lakes extracted — exactly matches CWC's own published count.
- 5/5** sample needs allocated to the correct provider.
- node_log** gives a step-by-step agent trace for every run.

Roadmap

- Live CWC river levels & IMD bulletins instead of derived alerts.
- Verified resource coordinates → true door-to-door routing.
- SMS / WhatsApp dispatch behind the same approval gate.
- Disaster-specific sub-agents on the already-wired disaster_type_router edge.
- Real-time volunteer availability sync.

ResQ · Retrieval finds the options. Deterministic scoring allocates them. A local LLM explains them. A human approves them.

AI does the triage math in seconds — people keep the authority. Thank you.